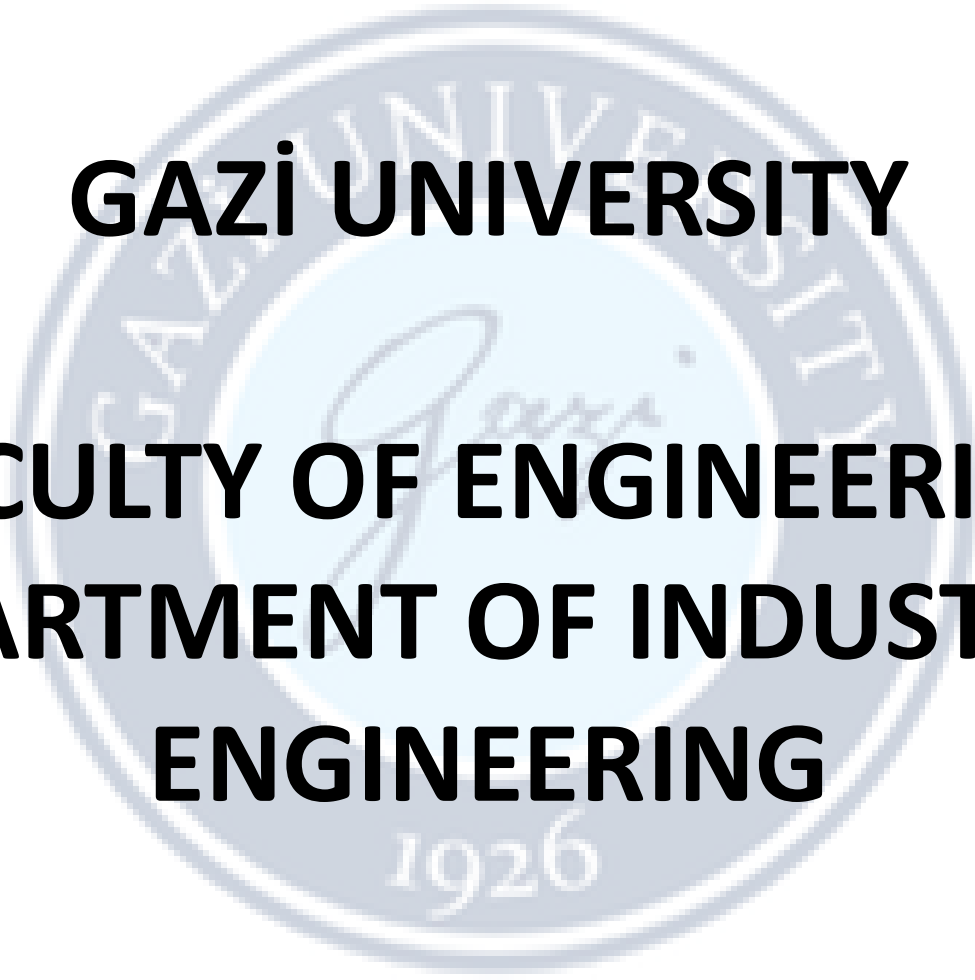


The logo of Gazi University is a circular seal. It features the university's name in Turkish, 'GAZİ ÜNİVERSİTESİ', around the top inner edge and the year '1926' at the bottom. In the center is a stylized signature of 'Gazi'.

GAZİ UNIVERSITY

FACULTY OF ENGINEERING

DEPARTMENT OF INDUSTRIAL ENGINEERING

Lecturer : Dr Ercan Ezin

IE326-DATABASE MANAGEMENT SYSTEMS

WEEK 14:EXAMPLE DB DESIGN/AI-AIDED DESIGN

PHYSICAL DESIGN EXAMPLE



PHYSICAL DESIGN

ER DIAGRAM

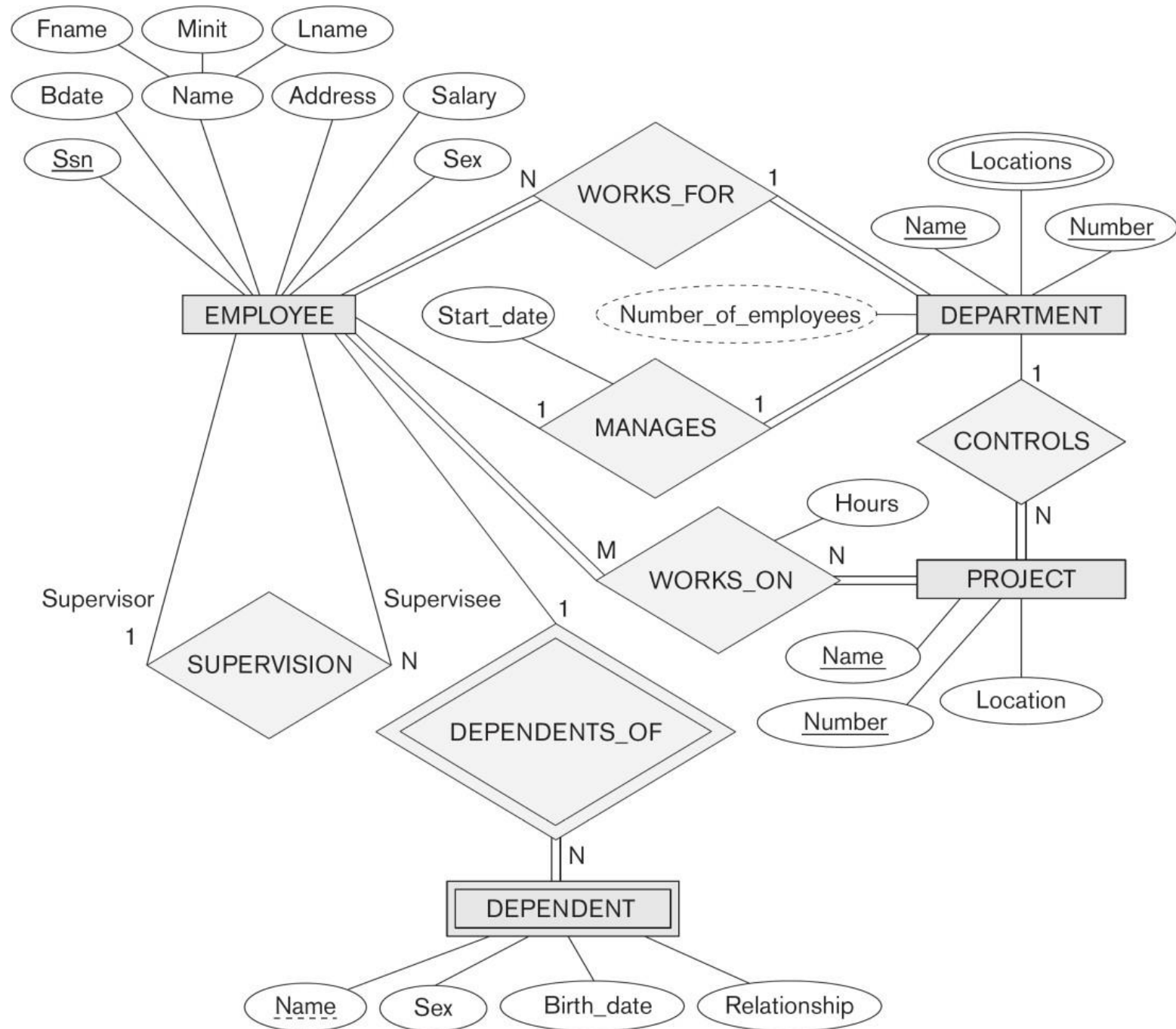


Table Creation

```
-- =====  
-- DROP TABLES  
-- =====  
  
DROP TABLE IF EXISTS works_on CASCADE;  
DROP TABLE IF EXISTS dependent CASCADE;  
DROP TABLE IF EXISTS employee CASCADE;  
DROP TABLE IF EXISTS project CASCADE;  
DROP TABLE IF EXISTS department_locations CASCADE;  
DROP TABLE IF EXISTS department CASCADE;
```

Table Creation

```
-- =====  
-- TABLE: DEPARTMENT  
-- =====  
  
CREATE TABLE department (  
    dnumber INTEGER PRIMARY KEY,  
    dname VARCHAR(50) NOT NULL UNIQUE,  
    mgr_ssn CHAR(9),  
    mgr_start_date DATE,  
    number_of_employees INTEGER DEFAULT 0  
);
```

Update

```
-- Department manager is an employee  
ALTER TABLE department  
ADD CONSTRAINT fk_department_manager  
FOREIGN KEY (mgr_ssn)  
REFERENCES employee(ssn);
```

```
-- =====  
-- TABLE: EMPLOYEE  
-- =====  
  
CREATE TABLE employee (  
    ssn CHAR(9) PRIMARY KEY,  
    fname VARCHAR(30) NOT NULL,  
    minit CHAR(1),  
    lname VARCHAR(30) NOT NULL,  
    bdate DATE,  
    address VARCHAR(150),  
    sex CHAR(1) CHECK (sex IN ('M', 'F')),  
    salary NUMERIC(10,2) CHECK (salary >= 0),  
    super_ssn CHAR(9),  
    dno INTEGER NOT NULL,  
  
    CONSTRAINT fk_employee_department  
        FOREIGN KEY (dno)  
        REFERENCES department(dnumber),  
  
    CONSTRAINT fk_employee_supervisor  
        FOREIGN KEY (super_ssn)  
        REFERENCES employee(ssn)  
);
```

Table Creation

```
-- =====
-- TABLE: DEPARTMENT_LOCATIONS
-- Multivalued attribute: Locations
-- =====

CREATE TABLE department_locations (
    dnumber INTEGER NOT NULL,
    dlocation VARCHAR(50) NOT NULL,

    PRIMARY KEY (dnumber, dlocation),

    CONSTRAINT fk_department_locations_department
        FOREIGN KEY (dnumber)
        REFERENCES department(dnumber)
        ON DELETE CASCADE
);
```

```
-- =====
-- TABLE: PROJECT
-- =====

CREATE TABLE project (
    pnumber INTEGER PRIMARY KEY,
    pname VARCHAR(50) NOT NULL UNIQUE,
    plocation VARCHAR(50),
    dnum INTEGER NOT NULL,

    CONSTRAINT fk_project_department
        FOREIGN KEY (dnum)
        REFERENCES department(dnumber)
);
```

Table Creation

```
-- =====
-- TABLE: WORKS_ON
-- M:N relationship between EMPLOYEE and PROJECT
-- Attribute: Hours
-- =====

CREATE TABLE works_on (
    essn CHAR(9) NOT NULL,
    pno INTEGER NOT NULL,
    hours NUMERIC(4,1) CHECK (hours >= 0),

    PRIMARY KEY (essn, pno),

    CONSTRAINT fk_works_on_employee
        FOREIGN KEY (essn)
        REFERENCES employee(ssn)
        ON DELETE CASCADE,

    CONSTRAINT fk_works_on_project
        FOREIGN KEY (pno)
        REFERENCES project(pnumber)
        ON DELETE CASCADE
);
```

```
-- =====
-- TABLE: DEPENDENT
-- Weak entity dependent on EMPLOYEE
-- =====

CREATE TABLE dependent (
    essn CHAR(9) NOT NULL,
    dependent_name VARCHAR(50) NOT NULL,
    sex CHAR(1) CHECK (sex IN ('M', 'F')),
    birth_date DATE,
    relationship VARCHAR(30),

    PRIMARY KEY (essn, dependent_name),

    CONSTRAINT fk_dependent_employee
        FOREIGN KEY (essn)
        REFERENCES employee(ssn)
        ON DELETE CASCADE
);
```

DATA INSERTION

— DEPARTMENTS

```
INSERT INTO department (dnumber, dname)
VALUES
```

```
(1, 'Executive Office'),  
(2, 'Research and Development'),  
(3, 'Software Engineering')
```

-- EMPLOYEES

```
INSERT INTO employee (
    ssn, fname, minit, lname, bdate, address, sex, salary, super_ssn, dno
)
```

```
VALUES
('100000001', 'Kemal', 'A', 'Yılmaz', '1978-03-12', 'Levent, İstanbul, Türkiye', 'M', 145000, NULL, 1),
('100000002', 'Elif', 'D', 'Kaya', '1983-07-21', 'Kadıköy, İstanbul, Türkiye', 'F', 128000, '100000001', 2),
('100000003', 'Ahmet', 'M', 'Demir', '1986-11-05', 'Çankaya, Ankara, Türkiye', 'M', 118000, '100000001', 3),
('100000004', 'Zeynep', 'S', 'Çelik', '1990-01-18', 'Beşiktaş, İstanbul, Türkiye', 'F', 112000, '100000002', 4),
('100000005', 'Mehmet', 'K', 'Şahin', '1984-09-09', 'Bornova, İzmir, Türkiye', 'M', 115000, '100000003', 5).
```

-- ASSIGN DEPARTMENT MANAGERS

```
UPDATE department SET mgr_ssn = '100000001', mgr_start_date = '2018-01-01' WHERE dnumber = 1;
UPDATE department SET mgr_ssn = '100000002', mgr_start_date = '2019-03-15' WHERE dnumber = 2;
UPDATE department SET mgr_ssn = '100000003', mgr_start_date = '2019-04-01' WHERE dnumber = 3;
UPDATE department SET mgr_ssn = '100000004', mgr_start_date = '2020-02-10' WHERE dnumber = 4;
UPDATE department SET mgr_ssn = '100000005', mgr_start_date = '2020-06-20' WHERE dnumber = 5;
```

-- DEPARTMENT LOCATIONS

```
INSERT INTO department_locations (dnumber, dlocation)
VALUES
```

```
(1, 'İstanbul'),
(1, 'London'),
(2, 'İstanbul'),
(2, 'Berlin'),
(3, 'Ankara'),
(2, 'Amsterdam')
```

```
-- WORKS_ON
```

```
INSERT INTO works_on (essn, pno, hours)
VALUES
```

```
( '1000000001', 2014, 8.0 ),
( '1000000001', 2019, 6.0 ),
( '1000000002', 2019, 24.0 ),
```

-- DEPENDENTS

```
INSERT INTO dependent (
    essn, dependent_name, sex, birth_date, relationship
)
VALUES
```

```
('100000001', 'Derya', 'F', '1980-05-10', 'Spouse'),
('100000001', 'Eren', 'M', '2010-09-14', 'Son'),
('100000002', 'Mina', 'F', '2014-03-22', 'Daughter'),
('100000003', 'Defne', 'F', '2016-07-09', 'Daughter'),
```

-- PROJECTS

```
INSERT INTO project (pnumber, pname, plocation, dnum)
VALUES
```

```
(2001, 'Anatolia AI Platform', 'İstanbul', 4),
(2002, 'European Cloud Migration', 'Berlin', 6),
(2003, 'Secure Banking Gateway', 'London', 5),
(2004, 'Multilingual E-Commerce System', 'İstanbul', 3),
(2005, 'Customer Success Portal', 'Madrid', 12)
```

DATA INSERTION

Please download following file to insert the data into the tables you created.

<https://gazi-end-muh.github.io/dbms/slides/week14.sql>

You must copy and paste the file in the sql studio and run it once.

VERIFY RECORDS

1st: Let's count total number of record on every table with the following code:

```
-- =====  
-- VERIFY COUNTS  
-- =====  
  
SELECT 'department' AS table_name, COUNT(*) AS record_count FROM department  
UNION ALL  
SELECT 'employee', COUNT(*) FROM employee  
UNION ALL  
SELECT 'department_locations', COUNT(*) FROM department_locations  
UNION ALL  
SELECT 'project', COUNT(*) FROM project  
UNION ALL  
SELECT 'works_on', COUNT(*) FROM works_on  
UNION ALL  
SELECT 'dependent', COUNT(*) FROM dependent;
```

ANSWER QUESTIONS

Question 1 — Department Salary Analysis

Write an SQL query to list:

- department name
- number of employees in the department
- average salary of employees in that department

Sort the result by average salary in descending order.

Expected Concepts:

- JOIN
- GROUP BY
- aggregate functions (COUNT, AVG)
- ORDER BY

Question 2 — Employees Working on Multiple Projects

Write an SQL query to find employees who work on more than one project.

Display:

- employee SSN
- full name (fname + lname)
- number of projects worked on

Sort the result by project count in descending order.

Expected Concepts:

- GROUP BY
- HAVING
- aggregate functions
- joins between employee and works_on

EXPECTED OUTPUT

Q1:

department_name	number_of_employees	average_salary	
Executive Office	1	145000.000000000000	
Research and Development	1	128000.000000000000	
Corporate Strategy	1	121000.000000000000	
Operations	1	119000.000000000000	
Cybersecurity	2	115500.000000000000	
Finance	2	114000.000000000000	
Data and AI	1	112000.000000000000	
Software Engineering	2	107500.000000000000	
Sales Europe	2	102500.000000000000	
Product Management	2	102500.000000000000	
Legal and Compliance	2	101500.000000000000	
Cloud Infrastructure	2	100000.000000000000	
Marketing	2	97500.000000000000	
Human Resources	1	96000.000000000000	
Business Intelligence	2	96000.000000000000	
Design and UX	2	93000.000000000000	
Quality Assurance	2	92500.000000000000	
Customer Success	1	90000.000000000000	
Procurement	1	86000.000000000000	

Q2:

ssn	full_name	project_count
100000007	Can Arslan	2
100000004	Zeynep Çelik	2
100000011	Deniz Yıldız	2
100000002	Elif Kaya	2
100000015	Okan Erdoğan	2
100000003	Ahmet Demir	2
100000018	Sophie Dubois	2
100000005	Mehmet Şahin	2
100000014	İrem Güneş	2
100000001	Kemal Yılmaz	2
100000006	Ayşe Aydın	2
100000012	Ece Doğan	2

ANSWERS

-- Q1: Department Salary Analysis

```
SELECT
    d.dname AS department_name,
    COUNT(e.ssn) AS number_of_employees,
    AVG(e.salary) AS average_salary
FROM department d
JOIN employee e
    ON d.dnumber = e.dno
GROUP BY d.dnumber, d.dname
ORDER BY average_salary DESC;
```

-- Q2: Employees Working on Multiple Projects

```
SELECT
    e.ssn,
    e.fname || ' ' || e.lname AS full_name,
    COUNT(w.pno) AS project_count
FROM employee e
JOIN works_on w
    ON e.ssn = w.essn
GROUP BY e.ssn, e.fname, e.lname
HAVING COUNT(w.pno) > 1
ORDER BY project_count DESC;
```

ANSWER QUESTIONS

Question 3 — Highest Paid Employee per Department

Write an SQL query to find the employee(s) with the highest salary in each department.

Display:

- department name
- employee full name
- salary

If multiple employees share the highest salary in the same department, display all of them.

Expected Concepts:

- correlated subquery or CTE
- aggregation with MAX
- joins
- handling ties correctly

Question 4 — Projects With No Assigned Employees

Write an SQL query to list all projects that currently have no employees assigned.

Display:

- project number
- project name
- department name responsible for the project

Expected Concepts:

- LEFT JOIN
- NULL checking
- joins across multiple tables
- understanding optional participation

EXPECTED OUTPUT

Q3:

department_name	full_name	salary
Business Intelligence	Okan Erdoğan	99000.00
Cloud Infrastructure	Marco Bianchi	102000.00
Corporate Strategy	Andreas Müller	121000.00
Customer Success	Sofia García	90000.00
Cybersecurity	Erik Johansson	116000.00
Data and AI	Zeynep Çelik	112000.00
Design and UX	Lucas van Dijk	99000.00
Executive Office	Kemal Yılmaz	145000.00
Finance	Emma Wilson	125000.00
Human Resources	Merve Öztürk	96000.00
Legal and Compliance	Oliver Brown	111000.00
Marketing	Sophie Dubois	106000.00
Operations	Pierre Martin	119000.00
Procurement	Maja Kowalska	86000.00
Product Management	Can Arslan	104000.00
Quality Assurance	Emre Aslan	94000.00
Research and Development	Elif Kaya	128000.00
Sales Europe	Anna Schneider	108000.00
Software Engineering	Ahmet Demir	118000.00

Q4:

pnumber	pname	department_name
999	NewAutomationSystem	Executive Office

ANSWERS

-- Q3: Highest Paid Employee per Department

```
SELECT
    d.dname AS department_name,
    e.fname || ' ' || e.lname AS full_name,
    e.salary
FROM employee e
JOIN department d
    ON e.dno = d.dnumber
WHERE e.salary = (
    SELECT MAX(e2.salary)
    FROM employee e2
    WHERE e2.dno = e.dno
)
ORDER BY d.dname, e.salary DESC;
```

-- Q4: Projects With No Assigned Employees

```
SELECT
    p.pnumber,
    p.pname,
    d.dname AS department_name
FROM project p
JOIN department d
    ON p.dnum = d.dnumber
LEFT JOIN works_on w
    ON p.pnumber = w.pno
WHERE w.pno IS NULL
ORDER BY p.pnumber;
```

Now you need to calculate your attendance

Download and insert the data from following address.

<https://gazi-end-muh.github.io/dbms/slides/week14-attendance.sql>

ANSWER FOLLOWING QUESTIONS

Question 1 — Total Attendance Count

Write an SQL query to display the total number of weeks attended by each student.

Your query should:

- display student number, first-name initial, and last-name initial,
- calculate the total attendance count for each student,
- include students who never attended any class.



Expected output

student_no	fname	lname	total_attendance
ELL000001	Y	A	0
ELL000002	I	K	7
ELL000003	H	S	5
ELL000004	D	Z	8
ELL000005	E	C	9
ELL000006	S	K	9
ELL000007	H	U	7

Question 2 — Attendance Percentage Calculation

Write an SQL query to calculate the attendance percentage of each student.

Your query should:

- display total attended weeks and total class weeks,
- calculate attendance percentage with 2 decimal precision,
- sort results according to student number.



Expected output

student_no	fname	lname	total_attendance	total_weeks_with_class	attendance_percentage
ELL000001	Y	A	0	10	0.00
ELL000002	I	K	7	10	70.00
ELL000003	H	S	5	10	50.00
ELL000004	D	Z	8	10	80.00
ELL000005	E	C	9	10	90.00
ELL000006	S	K	9	10	90.00
ELL000007	H	U	7	10	70.00

ANSWER FOLLOWING QUESTIONS

Question 3 — Attendance PASS / FAIL Evaluation

According to course regulations, students must attend at least 70% of the weeks where classes were held.

If the calculated minimum attendance requirement is decimal (for example 8.4 or 8.7), round it down using `FLOOR()`.

Write an SQL query that:

- calculates each student's attendance percentage,
- determines the minimum required attendance,
- displays whether the student PASSES or FAILS according to attendance rules.

student_no	fname	lname	total_attendance	total_weeks_with_class	minimum_required_attendance	attendance_percentage	attendance_status
0000000001	A	E	6	10	7	60.00	FAIL
0000000004	O	Ç	6	10	7	60.00	FAIL
0000000002	U	K	6	10	7	60.00	FAIL
0000000007	M	H	6	10	7	60.00	FAIL
0000000003	H	Ş	5	10	7	50.00	FAIL

Expected output

NOTE: This question is extra difficult, you should do some research before attempting to solve it.

ANSWERS

Q1:

```
SELECT
    s.student_no,
    s.fname,
    s.lname,
    COUNT(ar.week_no) AS total_attendance
FROM student s
LEFT JOIN attendance_record ar
    ON s.student_no = ar.student_no
GROUP BY
    s.student_no,
    s.fname,
    s.lname
ORDER BY s.student_no;
```

Q2:

```
SELECT
    s.student_no,
    s.fname,
    s.lname,
    COUNT(ar.week_no) AS total_attendance,

    (
        SELECT COUNT(DISTINCT week_no)
        FROM attendance_record
    ) AS total_weeks_with_class,

    ROUND(
        COUNT(ar.week_no) * 100.0 /
        (
            SELECT COUNT(DISTINCT week_no)
            FROM attendance_record
        ),
        2
    ) AS attendance_percentage

FROM student s
LEFT JOIN attendance_record ar
    ON s.student_no = ar.student_no

GROUP BY
    s.student_no,
    s.fname,
    s.lname

ORDER BY s.student_no;
```

ANSWERS

Q3:

```
SELECT
    s.student_no,
    s.fname,
    s.lname,

    COUNT(ar.week_no) AS total_attendance,

    (
        SELECT COUNT(DISTINCT week_no)
        FROM attendance_record
    ) AS total_weeks_with_class,

    FLOOR(
        (
            SELECT COUNT(DISTINCT week_no)
            FROM attendance_record
        ) * 0.70
    ) AS minimum_required_attendance,

    ROUND(
        COUNT(ar.week_no) * 100.0 /
        (
            SELECT COUNT(DISTINCT week_no)
            FROM attendance_record
        ),
        2
    ) AS attendance_percentage,

    CASE
        WHEN COUNT(ar.week_no) >= FLOOR(
            (
                SELECT COUNT(DISTINCT week_no)
                FROM attendance_record
            ) * 0.70
        )
        THEN 'PASS'

        ELSE 'FAIL'
    END AS attendance_status

FROM student s
LEFT JOIN attendance_record ar
    ON s.student_no = ar.student_no

GROUP BY
    s.student_no,
    s.fname,
    s.lname

ORDER BY
    attendance_status,
    attendance_percentage DESC,
    s.student_no;
```

THE END



CHECK OUT COURSE WEBSITE: [HTTPS://GAZI-END-MUH.GITHUB.IO](https://GAZI-END-MUH.GITHUB.IO)

GOT ANY QUESTIONS LATER?

SEND ME AN EMAIL: DR.ERCAN.EZIN@GMAIL.COM